

ML Assignment 1 — Bike Sharing Demand Prediction

Data Loading

```
[2]: import sys
print(sys.executable)

/home/cloud/anaconda3/bin/python

[4]: !conda install -y scikit-learn scipy numpy

Channels:
- defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

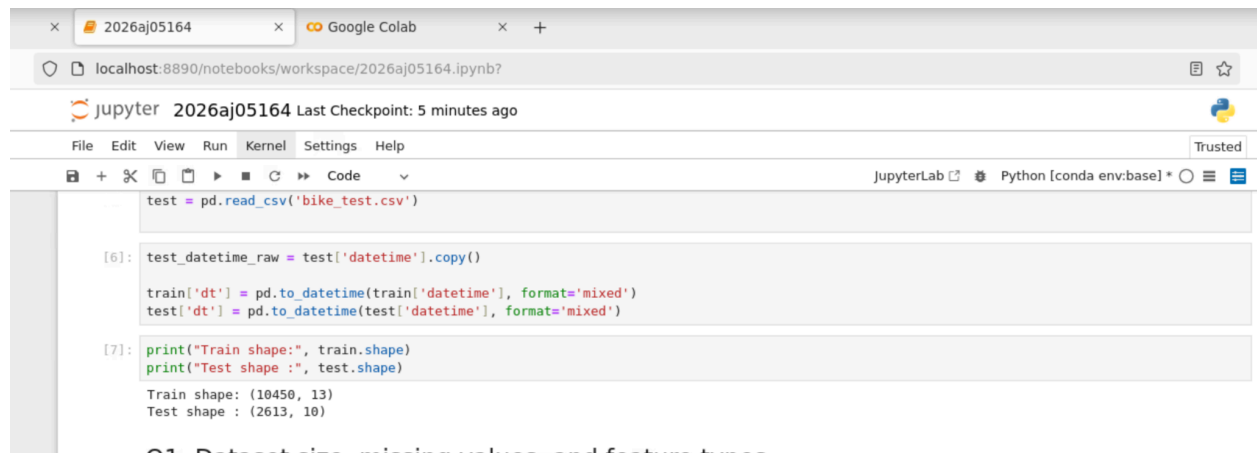
# All requested packages already installed.

[5]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso

sns.set_style('whitegrid')
pd.set_option('display.width', 120)

train = pd.read_csv('bike_train.csv')
test = pd.read_csv('bike_test.csv')
```



The screenshot shows a JupyterLab interface with a browser window at the top displaying the URL `localhost:8890/notebooks/workspace/2026aj05164.ipynb?`. Below the browser, the JupyterLab header shows the notebook name `2026aj05164` and the last checkpoint time `Last Checkpoint: 5 minutes ago`. The interface includes a menu bar with `File`, `Edit`, `View`, `Run`, `Kernel`, `Settings`, and `Help`. A toolbar with icons for file operations and execution is visible. The main area contains a code editor with the following code:

```
test = pd.read_csv('bike_test.csv')

[6]: test_datetime_raw = test['datetime'].copy()

train['dt'] = pd.to_datetime(train['datetime'], format='mixed')
test['dt'] = pd.to_datetime(test['datetime'], format='mixed')

[7]: print("Train shape:", train.shape)
print("Test shape :", test.shape)

Train shape: (10450, 13)
Test shape : (2613, 10)
```

The output of the code is displayed below the code cells, showing the shapes of the `train` and `test` datasets.

2026aj05164 Google Colab

localhost:8890/notebooks/workspace/2026aj05164.ipynb?

Jupyter 2026aj05164 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Trusted

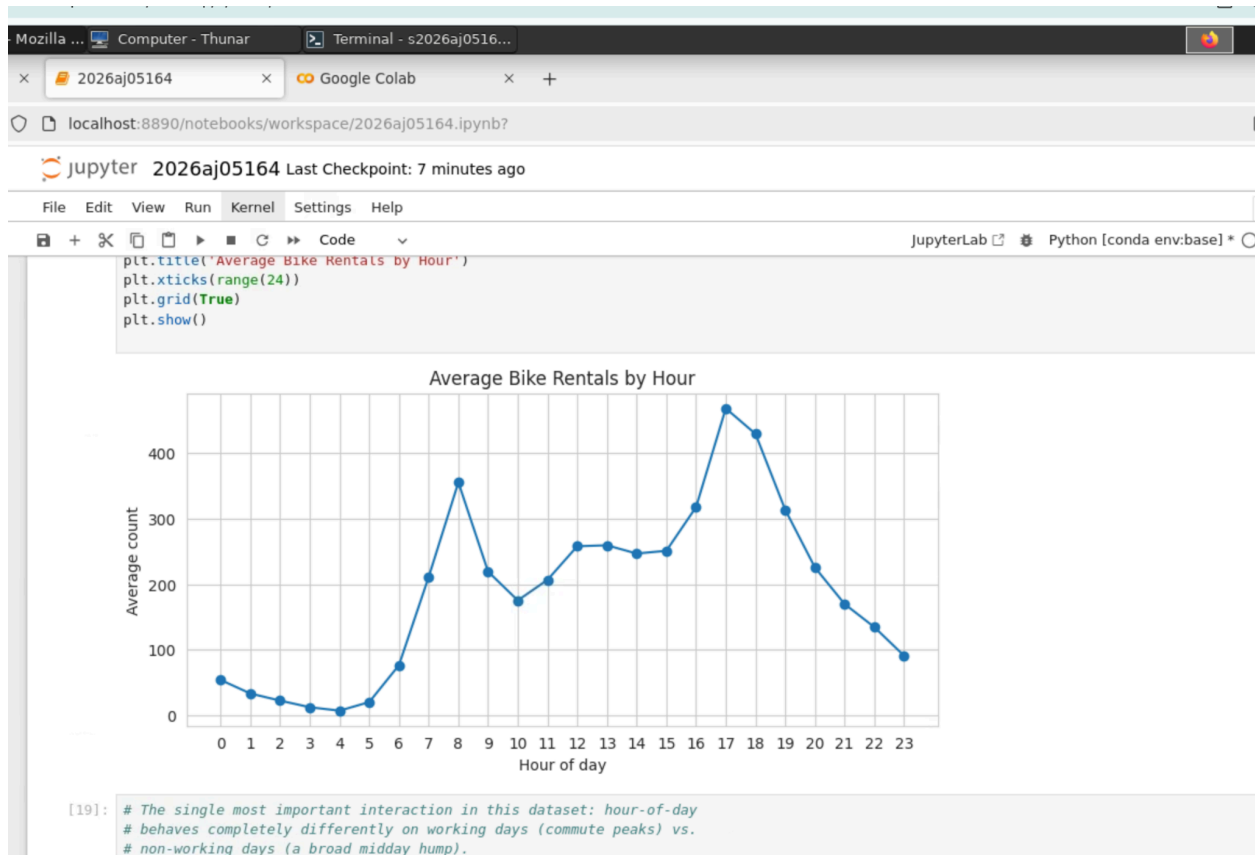
JupyterLab Python [conda env:base]

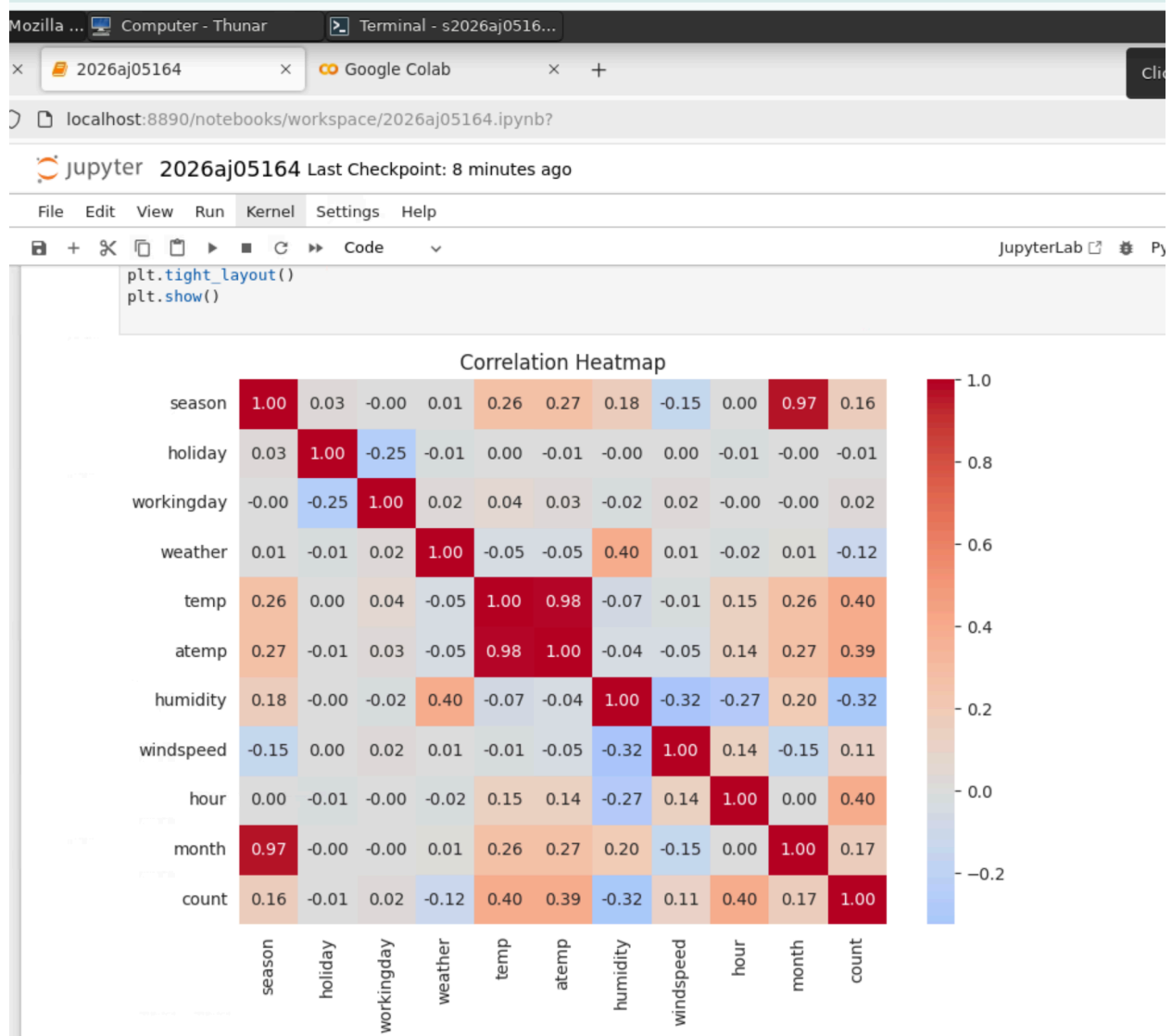
```
Training shape: (10450, 13)
Test shape: (2613, 10)

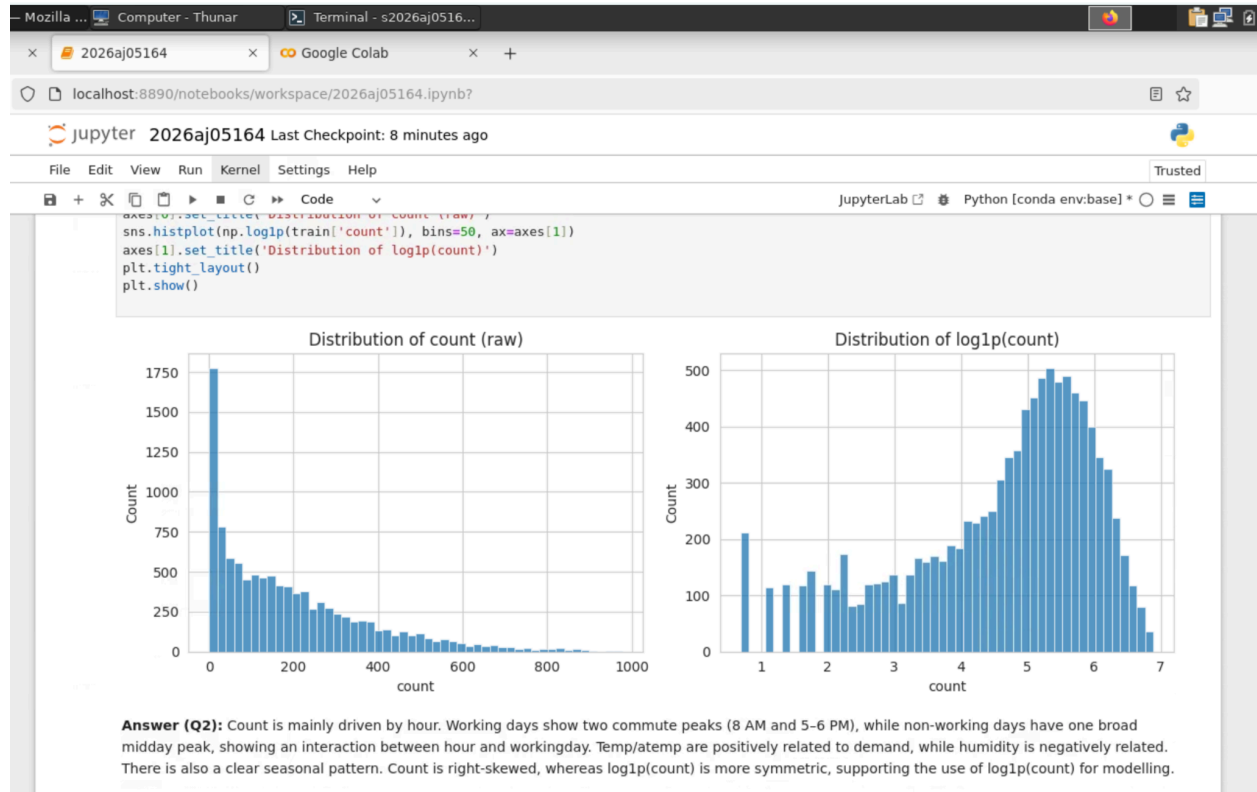
Missing values (train):
datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
casual      0
registered  0
count       0
dt          0
dtype: int64

Missing values (test):
datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
dt          0
dtype: int64

Training data types:
datetime    object
season      int64
holiday     int64
workingday  int64
weather     int64
temp        float64
atemp       float64
humidity    int64
windspeed   float64
casual      int64
registered  int64
```







2026aj05164 Google Colab

localhost:8890/notebooks/workspace/2026aj05164.ipynb?

Jupyter 2026aj05164 Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help Truste

JupyterLab Python [conda env:base] * ○ ≡

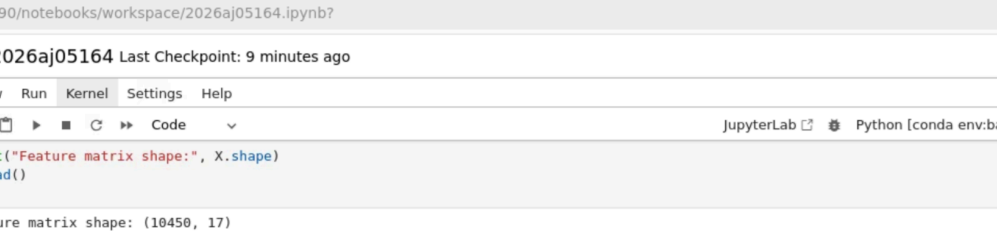
Answer (Q2): Count is mainly driven by hour. Working days show two commute peaks (8 AM and 5-6 PM), while non-working days have one broad midday peak, showing an interaction between hour and workingday. Temp/atemp are positively related to demand, while humidity is negatively related. There is also a clear seasonal pattern. Count is right-skewed, whereas log1p(count) is more symmetric, supporting the use of log1p(count) for modelling.

Q3. Which variables are likely to be most informative?

```
[27]: numeric_cols = train.select_dtypes(include='number')
      correlation = numeric_cols.corr()['count'].sort_values(ascending=False)
      print(correlation)
```

```
count      1.000000
registered  0.970108
casual      0.691971
temp        0.396451
atemp       0.390642
season      0.160333
windspeed   0.105318
workingday   0.017361
holiday     -0.005615
weather     -0.124402
humidity    -0.316607
Name: count, dtype: float64
```

Answer (Q3): Hour has the strongest linear correlation with count, followed by temp/atemp, while humidity is negatively correlated. Month/season show a moderate seasonal effect. Workingday, weekday, and holiday have little linear correlation individually, but their interaction with hour makes them important. Year also has low individual correlation but reflects the growth in bike-sharing demand from 2011 to 2012.



The screenshot shows a JupyterLab interface. At the top, there's a browser address bar with 'localhost:8890/notebooks/workspace/2026aj05164.ipynb?'. Below it, the JupyterLab header shows 'jupyter 2026aj05164' and 'Last Checkpoint: 9 minutes ago'. The main toolbar includes 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. A 'Trusted' badge is visible on the right. The code cell contains the following Python code:

```
print("Feature matrix shape:", X.shape)
X.head()
```

The output of the code is displayed below the cell:

```
Feature matrix shape: (10450, 17)
```

```
[31]:
```

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	year	month	day	hour	weekday	day	hour_work	hour_we
0	3	0	0	1	28.70000	33.335000	79	6.003200	2012	7	15	7	6	197	7_0	
1	3	0	1	1	33.62000	37.880000	46	15.001300	2012	8	14	15	1	227	15_1	
2	1	0	0	1	10.66000	12.880000	60	15.001300	2011	2	6	6	6	37	6_0	
3	2	0	0	2	26.42506	30.566166	61	9.512288	2012	5	6	17	6	127	17_0	
4	1	0	1	1	9.84000	12.120000	56	8.998100	2012	1	9	2	0	9	2_1	

rayogshala.bits-pilani.ac.in/web-rdp/#/client/MTM1MzIAYwBwb3N0Z3Jlc3Fs?token=06529DB595A3D181E04D9B4A36024636537E965A792CB96272651AD0FFB85DD7

05164 — Mozilla ... Computer - Thunar Terminal - s2026aj0516...

2026aj05164 Google Colab

localhost:8890/notebooks/workspace/2026aj05164.ipynb?

Jupyter 2026aj05164 Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help

Code

JupyterLab Python [conda env:base] *

```
print("Ridge, polynomial degree 2")
print(ridge2_sweep.to_string(index=False))

best_alpha_ridge2 = select_alpha_1se(
    ridge2_sweep['alpha'].tolist(),
    ridge2_sweep['cv_RMSLE'].tolist(),
    ridge2_sweep['cv_std'].tolist()
)
print("\nSelected alpha (1-SE rule):", best_alpha_ridge2)
```

```
Ridge, polynomial degree 2
alpha train_RMSLE cv_RMSLE cv_std
1 0.377749 0.389504 0.008996
3 0.378407 0.389269 0.008781
5 0.379148 0.389428 0.008704
7 0.379930 0.389778 0.008686
9 0.380773 0.390280 0.008694
12 0.382163 0.391255 0.008721
15 0.383711 0.392473 0.008757
20 0.386616 0.394930 0.008879
30 0.393508 0.401173 0.009166
```

```
Selected alpha (1-SE rule): 20
```

```
[54]: ridge3_sweep = sweep_ridge(degree=3)
```

```
print("Ridge, polynomial degree 3")
```

```
train_RMSLE cv_RMSLE cv_std
1 0.377749 0.389504 0.008996
3 0.378407 0.389269 0.008781
5 0.379148 0.389428 0.008704
7 0.379930 0.389778 0.008686
9 0.380773 0.390280 0.008694
12 0.382163 0.391255 0.008721
15 0.383711 0.392473 0.008757
20 0.386616 0.394930 0.008879
30 0.393508 0.401173 0.009166
```

```
[48]: tr_lin, va_lin, sd_lin = cv_rmsle(make_linear_pipeline, X, y)
```

```
print(f"Engineered Log-Linear (one-hot, no polynomial):")
print(f" train RMSLE = {tr_lin:.4f} CV RMSLE = {va_lin:.4f} +/- {sd_lin:.4f}")
```

```
Engineered Log-Linear (one-hot, no polynomial):
train RMSLE = 0.3870 CV RMSLE = 0.3986 +/- 0.0080
```

```
[50]: alpha_grid_ridge = [1, 3, 5, 7, 9, 12, 15, 20, 30]
```

```
def select_alpha_1se(alphae, cv_means, cv_std):
```

2026aj05164Google Colablocalhost:8890/notebooks/workspace/2026aj05164.ipynb?

jupyter 2026aj05164 Last Checkpoint: 10 minutes ago

File Edit View Run Kernel Settings Help

JupyterLab Python [conda env:base] *

	9	0.380773	0.390280	0.008694
	12	0.382163	0.391255	0.008721
	15	0.383711	0.392473	0.008757
	20	0.386616	0.394930	0.008879
	30	0.393508	0.401173	0.009166

Selected alpha (1-SE rule): 20

```
[54]: ridge3_sweep = sweep_ridge(degree=3)

print("Ridge, polynomial degree 3")
print(ridge3_sweep.to_string(index=False))

best_alpha_ridge3 = select_alpha_1se(
    ridge3_sweep['alpha'].tolist(),
    ridge3_sweep['cv_RMSLE'].tolist(),
    ridge3_sweep['cv_std'].tolist()
)
print("\nSelected alpha (1-SE rule):", best_alpha_ridge3)
```

Ridge, polynomial degree 3

alpha	train_RMSLE	cv_RMSLE	cv_std
1	0.372080	0.385534	0.008465
3	0.372967	0.385294	0.008459
5	0.373906	0.385528	0.008455
7	0.374874	0.385986	0.008488
9	0.375857	0.386562	0.008535
12	0.377412	0.387635	0.008597
15	0.379095	0.388918	0.008663
20	0.382158	0.391447	0.008795
30	0.389247	0.397804	0.009115

Selected alpha (1-SE rule): 20

```
[55]: lasso_alpha_grid = [0.0005, 0.001, 0.002, 0.005]
```

titled0.ipynb
SampleSubmis
Comparing bik
Explain Every
Inbox (114) - 2
BITS Lab Man
Lab - 7976 - s
Share Email Ac
Untitled docum

j3.prayogshala.bits-pilani.ac.in/web-rdp/#/client/MTM1MzIAYwBwb3N0Z3Jlc3Fs?token=06529DB595A3D181E04D9B4A36024636537E965A792CB96272651AD0FFB85DD7

2026aj05164
Google Colab

localhost:8890/notebooks/workspace/2026aj05164.ipynb?

Jupyter
2026aj05164 Last Checkpoint: 10 minutes ago

File Edit View Run Kernel Settings Help

JupyterLab Python [conda env:base]

```

return pd.DataFrame(rows, columns=['alpha', 'train_RMSLE', 'cv_RMSLE', 'cv_std'])

lasso2_sweep = sweep_lasso(degree=2)

print("Lasso, polynomial degree 2")
print(lasso2_sweep.to_string(index=False))

lasso3_sweep = sweep_lasso(degree=3)

print("\nLasso, polynomial degree 3")
print(lasso3_sweep.to_string(index=False))

Lasso, polynomial degree 2
alpha train_RMSLE cv_RMSLE cv_std
0.0005 0.386712 0.394582 0.009599
0.0010 0.395906 0.401389 0.009555
0.0020 0.410820 0.414828 0.009373
0.0050 0.458584 0.462023 0.009071

Lasso, polynomial degree 3
alpha train_RMSLE cv_RMSLE cv_std
0.0005 0.383683 0.392163 0.009481
0.0010 0.393547 0.399256 0.009663
0.0020 0.407897 0.411979 0.009659
0.0050 0.454632 0.458158 0.009262

Observation (Q6): For degree-3 Ridge, CV RMSLE is lowest around alpha ≈ 3 (≈0.3853) and rises at higher alpha values, while training RMSLE keeps improving as alpha decreases, showing mild overfitting. The 1-standard-error rule selects the more stable alpha = 20. Degree-3 performs slightly better than degree-2, while Ridge is more stable than Lasso as alpha increases.

```

Q7. Model Comparison

The first two rows (Q5 raw-feature baselines) use the required 80/20 hold-out split. All subsequent models, including the no-polynomial engineered baseline, use 5-fold cross-validation, which provides a more reliable performance estimate and is used to select the final alpha and degree. Thus, CV results are the most reliable for model comparison.

```

[60]: best_ride2_row = ridge2_sweep.loc[ridge2_sweep['alpha'] == best_alpha_ride2].iloc[0]
best_ride3_row = ridge3_sweep.loc[ridge3_sweep['alpha'] == best_alpha_ride3].iloc[0]

```

```
best_lasso2_row['train_RMSLE'], best_lasso3_row['train_RMSLE'],
],
'Val / CV RMSLE': [
    rmsle_raw, rmsle_log, va_lin,
    best_ridge2_row['cv_RMSLE'], best_ridge3_row['cv_RMSLE'],
    best_lasso2_row['cv_RMSLE'], best_lasso3_row['cv_RMSLE'],
],
])
comparison['Gap (Val/CV - Train)'] = comparison['Val / CV RMSLE'] - comparison['Train RMSLE']
comparison = comparison.sort_values('Val / CV RMSLE').reset_index(drop=True)
comparison.index += 1
comparison.index.name = 'Rank'
print(comparison.to_string())

best_model_name = comparison.iloc[0]['Model']
print("\nBest model:", best_model_name)
```

Rank	Model	Train RMSE	Val / CV RMSE	Gap (Val/CV - Train)
1	Ridge, poly deg=3, alpha=20 [1-SE] (5-fold CV)	0.382158	0.391447	0.009289
2	Lasso, poly deg=3, alpha=0.0005 (5-fold CV)	0.383683	0.392163	0.008480
3	Lasso, poly deg=2, alpha=0.0005 (5-fold CV)	0.386712	0.394582	0.007870
4	Ridge, poly deg=2, alpha=20 [1-SE] (5-fold CV)	0.386616	0.394930	0.008313
5	Engineered Log-Linear, one-hot, no polynomial (5-fold CV)	0.386974	0.398575	0.011602
6	Simple Linear Regression, logp target (Q5, single split)	1.221970	1.221970	0.000000
7	Simple Linear Regression, raw target (Q5, single split)	1.421538	1.421538	0.000000

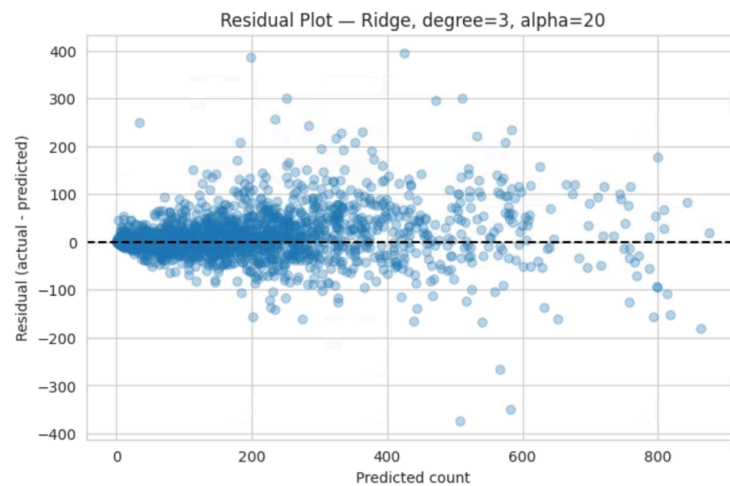
Best model: Ridge, poly deg=3, alpha=20 [1-SE] (5-fold CV)

Interpretation (Q7): Raw-feature linear models perform poorly (RMSLE ≈ 1.2 -1.4) because they cannot capture the strong time-of-day pattern. Adding calendar and interaction features greatly improves performance, reducing CV RMSLE to ≈ 0.399 , mainly due to `hour_work` and `hour_weekday` interactions. Polynomial weather features give a smaller further improvement, with **Ridge (degree 3, alpha = 20)** performing best at CV RMSLE ≈ 0.391 , closely followed by Lasso (degree 3). The train/CV gap remains below ≈ 0.012 , indicating good generalization with limited overfitting.

Q8. Residual Plot (Best Model)

```
fcat). best_degree = 2 if 1deg_21 in best_model else 2
```

```
print("Hold-out validation RMSLE for this plot:", rmsle(y_val, pred_best_val))
```



Hold-out validation RMSLE for this plot: 0.39012121777378217

Observation (Q8): Residuals are centered near zero but **fan out at higher predicted counts**, indicating heteroscedasticity. The model is more accurate for low-demand hours but tends to under-predict high-demand commute peaks. This reflects the limitation of a regularized linear model, which may not capture extreme combinations of conditions as well as tree-based or boosted models.

Mozilla ... Computer - Thunar Terminal - s2026aj0516...

2026aj05164 Google Colab

localhost:8890/notebooks/workspace/2026aj05164.ipynb?

jupyter 2026aj05164 Last Checkpoint: 11 minutes ago

File Edit View Run Kernel Settings Help

Code

```
[72]: X_test = make_features(test).drop(columns=['datetime'])

final_model = make_pipeline(best_degree, best_alpha, 'ridge')
final_model.fit(X, np.log1p(y))

test_pred = np.clip(np.expml(final_model.predict(X_test)), 0, None)
test_pred = np.round(test_pred).astype(int) # count is inherently a whole n

submission = pd.DataFrame({
    'datetime': test_datetime_raw,
    'count_predicted': test_pred
})

submission.to_csv('submission.csv', index=False)
print(submission.head())
print("\nShape:", submission.shape)
```

	datetime	count_predicted
0	05-06-2012 5:00	14
1	19-03-2011 19:00	121
2	02-04-2012 6:00	55
3	13-07-2012 20:00	356
4	09-11-2011 19:00	234

Shape: (2613, 2)

```
print("\nMissing values:")
print(submission.isna().sum())

print("\nNegative predictions:")
print((submission['count_predicted'] < 0).sum())
```

	datetime	count_predicted
0	05-06-2012 5:00	14
1	19-03-2011 19:00	121
2	02-04-2012 6:00	55
3	13-07-2012 20:00	356
4	09-11-2011 19:00	234

Shape:

(2613, 2)

Columns:

['datetime', 'count_predicted']

Missing values:

datetime 0

count_predicted 0

dtype: int64

Negative predictions:

0